

Tejiendo Redes: Arquitectura de Software Entre Hilos y Nodos

Red Distribuida de Puntos de Venta para el Operador GANA

Esteban Sánchez Viana

Corporación Escuela Tecnológica del Oriente

Programa de Ingeniería de Software

Arquitectura de Software

Unidad 2 — Mini proyecto: Patrones de Diseño y Arquitectura Distribuida

Edward Alfonso Villamizar Vallejo

12 de septiembre de 2026

Índice

Introducción.....	4
Descripción del Sistema Distribuido.....	5
Delimitación del Subsistema.....	5
Componentes Funcionales.....	5
Supuestos y Restricciones.....	6
Análisis y Planificación del Proyecto.....	6
Definición de Requisitos.....	6
Trazabilidad con la Unidad Anterior.....	9
Relevancia de la Comunicación Entre Nodos e Hilos.....	9
Metas del Sistema.....	10
Estrategia de Trabajo y Cronograma.....	11
Modelado de la Arquitectura Mediante UML.....	12
Diagrama de Componentes.....	12
Diagrama de Secuencia.....	13
Diagrama de Despliegue.....	14
Selección y Justificación de Patrones de Diseño.....	15
Criterio de Selección.....	15
Justificación de los Patrones Principales.....	16
Patrones Descartados.....	17
Implementación y Simulación del Prototipo.....	18
Alcance del Prototipo.....	18
Tecnologías y Estructura.....	18
Evidencia de los Patrones en el Código.....	19
Repositorio y Puesta en Marcha.....	19
Evaluación del Desempeño y Ajustes.....	20
Método de Medición.....	20

Escalabilidad Horizontal y Cuello de Botella.....	20
Ajuste Aplicado y Medición Posterior.....	20
Punto de Saturación de los Hilos.....	22
Pruebas de Robustez.....	24
Cumplimiento de las Metas.....	25
Conclusiones.....	26
Referencias.....	27

Introducción

Este informe presenta el diseño, la construcción y la evaluación de una arquitectura de software distribuida para la red de puntos de venta de GANA, un operador de juegos de suerte y azar que trabaja en el mercado colombiano. El caso se adapta con fines académicos, o sea, las cifras, los tiempos y los volúmenes que aparecen acá no corresponden a la operación real de la empresa, sino a un escenario construido para el ejercicio.

El trabajo continúa la propuesta que se entregó en la Unidad 1, donde se documentaron veinte requisitos funcionales, catorce no funcionales y ocho patrones de diseño con su respectiva trazabilidad. Ahora bien, aquella entrega se quedó en el plano de la propuesta, ya que los patrones se justificaron, pero no se midieron. Acá se retoma un subsistema concreto de esa arquitectura y se lleva al terreno de lo distribuido.

El subsistema elegido es la red de puntos de venta durante el cierre de un sorteo. Se escogió porque en él conviven tres condiciones al mismo tiempo. Primero, hay nodos separados físicamente, con conectividad intermitente, que es la definición práctica de un sistema distribuido. Segundo, cada punto de venta atiende a varios apostadores a la vez, así que el manejo de hilos aparece de forma natural. Y tercero, el cierre de sorteo concentra el pico de carga del negocio, es decir, es el momento en el que la escalabilidad y la robustez se pueden medir con números.

Algo que me gustaría aclarar es que, como lo que se entrega es un prototipo con sus mediciones, en este informe se reporta tanto lo que funcionó como lo que se quedó corto frente a la meta comprometida, ya que si uno esconde un resultado negativo el ejercicio de evaluación no sirve para nada.

El documento detalla la descripción del sistema, el análisis de requisitos y la planificación, el modelado en UML, la selección de patrones, la implementación del prototipo, la evaluación con métricas reales y, por último, las conclusiones.

Descripción del Sistema Distribuido

Delimitación del Subsistema

Como se explicó en el ejercicio de la Unidad 1, las operaciones de GANA dependen de dos canales de venta muy distintos entre sí. El digital, que consta de la aplicación web y móvil, exige disponibilidad permanente, y el físico, formado por una red amplia de puntos de venta repartidos por el territorio nacional, debe seguir funcionando aun cuando se cae la conexión a internet.

Para este caso, el enfoque es el canal físico. La razón es que ahí la intermitencia de red es una condición normal del día a día y no una excepción rara que se resuelva con un mensaje de error, y eso obliga a que la tolerancia a fallos viva en el diseño y no en la interfaz. De hecho, esa fue una de las conclusiones más claras del trabajo anterior, así que resulta interesante profundizar ahí.

Al sistema resultante se le puso el nombre Red Distribuida de Puntos de Venta, o RDPV para abreviar. Su propósito es recibir apuestas en varios nodos a la vez, mantener la coherencia del cupo asignado a cada punto de venta, sobrevivir a la desconexión del nodo central y liquidar los premios de forma asíncrona, sin frenar el registro de apuestas en el momento de mayor carga.

Componentes Funcionales

El sistema se descompuso en cinco tipos de nodo, cada uno con una responsabilidad clara. La Tabla 1 los presenta junto con su cardinalidad.

Tabla 1

Nodos que Componen la Red Distribuida

Nodo	Cantidad	Responsabilidad
Coordinador	1	Registro de nodos, validación de idempotencia, sello de cierre de sorteo y consolidación del estado global
Punto de venta	N configurable	Recepción concurrente de apuestas, control del cupo local y bitácora de salida ante desconexión
Bus de eventos	1	Cola persistente que desacopla el registro de apuestas de la liquidación de premios
Liquidador	M configurable	Consumo concurrente de eventos, cálculo de premios y emisión de compensaciones

Nodo	Cantidad	Responsabilidad
Monitor	1	Recolección de métricas de latencia, throughput y estado de los nodos

Nota. Elaboración propia. La cardinalidad de los nodos de punto de venta y de los liquidadores se define al levantar la red.

Hace falta explicar por qué existe el coordinador. Se podría pensar que los nodos se hablen directamente entre sí, pero eso haría crecer las rutas de comunicación de forma cuadrática y volvería inmanejable el control de duplicados. Al concentrar la coordinación en un solo punto se gana claridad, aunque, como se verá en la evaluación, también se paga un precio.

Supuestos y Restricciones

La red se simula sobre una sola máquina, pero cada nodo corre como un proceso independiente del sistema operativo y se comunica por sockets. Esto conserva lo importante, que son los espacios de memoria separados y la posibilidad real de que un mensaje falle, se pierda o llegue tarde. Con un conjunto de objetos llamándose en memoria no se habría podido demostrar nada de eso.

La persistencia se resolvió con archivos locales por nodo. No se usó un motor de base de datos porque introduciría una dependencia externa y desplazaría el foco desde lo distribuido hacia lo transaccional, que ya se había tratado en la unidad anterior. Además, así el entorno completo se levanta con un solo comando y sin instalar nada, lo que cumple el requisito RD-NF13.

Quedan fuera del alcance la interfaz de usuario, las pasarelas de pago reales y el canal digital.

Análisis y Planificación del Proyecto

Definición de Requisitos

Antes de dibujar un solo diagrama se escribieron los requisitos, ya que un diagrama que no se puede rastrear hasta una necesidad concreta termina justificándose hacia atrás. Se usó el método MoSCoW para priorizar, igual que en la Unidad 1, y se les asignó el prefijo RD para distinguirlos de los requisitos generales del sistema.

Debido a todo lo que hay que documentar en este caso, no se repitieron los veinte requisitos funcionales del trabajo anterior. Aquí se trabajó con un subconjunto acotado y propio de lo distribuido, formado por la comunicación entre nodos, la concurrencia mediante hilos, la tolerancia a la partición de red y la sincronización diferida. La Tabla 2 los recoge.

Tabla 2

Requisitos Funcionales del Sistema Distribuido

ID	Requisito	Prioridad
RD-F01	Registrar cada nodo ante el coordinador al iniciar y mantener la afiliación mediante latidos periódicos	Obligatorio
RD-F02	Detectar la caída de un nodo por ausencia de latidos y marcarlo como no disponible	Obligatorio
RD-F03	Atender varias solicitudes de apuesta de forma concurrente mediante un conjunto de hilos trabajadores	Obligatorio
RD-F04	Descotar el cupo del punto de venta bajo exclusión mutua entre los hilos del nodo	Obligatorio
RD-F05	Enviar cada apuesta con una clave de idempotencia generada en el nodo de origen	Obligatorio
RD-F06	Registrar la apuesta en una bitácora de salida local cuando el coordinador resulte inalcanzable	Obligatorio
RD-F07	Continuar recibiendo apuestas mientras el nodo permanece sin conexión con el coordinador	Obligatorio
RD-F08	Sincronizar la bitácora de salida al restablecerse la conexión, respetando el orden de generación	Obligatorio
RD-F09	Reconocer las claves repetidas y devolver el resultado original en lugar de crear una apuesta nueva	Obligatorio
RD-F10	Difundir el evento de cierre de sorteo desde el coordinador hacia todos los nodos suscritos	Obligatorio
RD-F11	Rechazar toda apuesta cuyo sello de tiempo sea posterior al cierre difundido	Obligatorio
RD-F12	Publicar en la cola de eventos cada apuesta confirmada para su liquidación posterior	Obligatorio
RD-F13	Consumir la cola mediante varios liquidadores sin procesar dos veces el mismo evento	Obligatorio
RD-F14	Emitir un evento de compensación que revierta la reserva cuando un paso posterior falle	Obligatorio
RD-F15	Interrumpir las llamadas hacia un destino que supere el umbral de fallos y degradar de forma controlada	Obligatorio
RD-F16	Consolidar y exponer el estado global de la red	Recomendable
RD-F17	Registrar por nodo las métricas de latencia, throughput y errores	Recomendable
RD-F18	Permitir la incorporación de nodos adicionales sin detener los que ya operan	Recomendable
RD-F19	Provocar fallos y particiones de red de forma deliberada para efectos de demostración	Recomendable

Nota. La prioridad se asignó con el método MoSCoW. El requisito RD-F19 no proviene del negocio sino del ejercicio académico, ya que sin la capacidad de romper el sistema a voluntad no habría forma de evidenciar su robustez. Elaboración propia.

Los requisitos no funcionales son los que de verdad condicionan la estructura, y por eso a cada uno se le asignó una métrica verificable. Un requisito sin métrica no se puede comprobar ni discutir, además es lo que después permite declarar si el prototipo pasó o falló. La Tabla 3 los presenta.

Tabla 3

Requisitos No Funcionales y sus Métricas de Verificación

ID	Categoría	Requisito	Métrica
RD-NF01	Rendimiento	Latencia de registro de una apuesta	Percentil 95 menor o igual a 300 ms
RD-NF02	Escalabilidad horizontal	Crecimiento del throughput al agregar nodos	Al menos 70 % del incremento lineal ideal de 1 a 4 nodos
RD-NF03	Escalabilidad vertical	Aprovechamiento del conjunto de hilos	Punto de saturación identificado y documentado
RD-NF04	Concurrencia	Integridad del cupo bajo acceso simultáneo	Cero sobregiros de cupo
RD-NF05	Tolerancia a particiones	Continuidad del nodo sin conexión	Cero apuestas rechazadas por la desconexión
RD-NF06	Consistencia eventual	Convergencia tras restablecer la conexión	Cero pérdidas y cero duplicados
RD-NF07	Recuperación	Tiempo de sincronización al volver en línea	Menor a 5 s para 500 apuestas acumuladas
RD-NF08	Detección de fallos	Latencia en detectar un nodo caído	Menor o igual a 3 latidos perdidos
RD-NF09	Aislamiento	Efecto de la caída de un nodo sobre el resto	Ninguna degradación medible en los demás
RD-NF10	Robustez	Comportamiento ante un destino degradado	Apertura del circuito antes de agotar los hilos
RD-NF11	Coherencia temporal	Efectividad del corte por cierre de sorteo	Cero apuestas admitidas tras el sello
RD-NF12	Observabilidad	Cobertura de la instrumentación	Todos los nodos reportando métricas
RD-NF13	Reproducibilidad	Puesta en marcha del entorno completo	Un solo comando, sin dependencias externas
RD-NF14	Portabilidad	Independencia del entorno de ejecución	Biblioteca estándar, sin servicios propietarios

Nota. Los umbrales de RD-NF01 y RD-NF02 se heredan de los requisitos RNF-01 y RNF-02 comprometidos en la Unidad 1, de modo que el prototipo pone a prueba lo que la propuesta original prometió. Elaboración propia.

Trazabilidad con la Unidad Anterior

Los requisitos distribuidos no se redactaron desde cero. Cada uno se desprende de un requisito funcional o no funcional que ya había quedado documentado en la unidad anterior, y la tabla siguiente deja explícita esa correspondencia.

Tabla 4

Origen de los Requisitos Distribuidos en la Unidad 1

Requisitos distribuidos	Origen	Relación
RD-F03, RD-F04, RD-NF01, RD-NF03, RD-NF04	RF-04, RNF-01	El registro de apuestas en punto de venta se lleva al plano concurrente
RD-F05 a RD-F09	RF-06, RNF-04	La operación sin conexión y su sincronización se implementan y se miden
RD-F10, RD-F11	RF-03, RF-10	El horario de cierre se convierte en un evento difundido a la red
RD-F12, RD-F13	RF-10, RF-11	La liquidación de premios pasa a procesamiento asíncrono concurrente
RD-F14	RNF-05	La consistencia entre servicios se resuelve por compensación
RD-F15, RD-NF10	RNF-04, RF-16	La protección ante destinos degradados se materializa en código
RD-F01, RD-F02, RD-F16, RD-F17	RNF-11	La observabilidad deja de ser un enunciado y se vuelve instrumentación
RD-F18, RD-NF02	RNF-02	El escalado por partes se demuestra con medición

Nota. Elaboración propia a partir del informe de la Unidad 1.

Relevancia de la Comunicación Entre Nodos e Hilos

Aunque los dos planos suelen nombrarse juntos, en realidad son problemas distintos y se resuelven de formas distintas, así que vale la pena separarlos.

Entre hilos, dentro de un mismo nodo, el problema es la memoria compartida. Varios hilos del punto de venta tocan el mismo contador de cupo, y sin exclusión mutua dos apuestas simultáneas pueden leer el mismo saldo disponible y descontarlo dos veces sobre la misma base. Eso produce un sobregiro que no salta como error. Simplemente deja un número equivocado, que es el peor tipo de falla porque es silenciosa. Acá la comunicación es implícita, ocurre a través del estado, y el costo de coordinar es un bloqueo que reduce el paralelismo. De ahí que el tamaño del

conjunto de hilos no sea un número que se elija al azar, ya que por debajo se desaprovecha la máquina y por encima los hilos pasan más tiempo esperando el candado que trabajando.

Entre nodos la situación cambia por completo, ya que no hay memoria compartida y la comunicación es explícita. Un mensaje viaja por la red y puede perderse, llegar tarde, llegar dos veces o no llegar nunca. La diferencia de fondo es que un hilo bloqueado eventualmente avanza, mientras que un nodo que no responde es indistinguible de un nodo lento. Por eso hacen falta mecanismos completamente diferentes: idempotencia, porque el reenvío es inevitable; detección por latidos, porque el silencio hay que interpretarlo; y compensación en lugar de reversión, porque no existe una transacción que abarque los dos extremos.

La guía de la unidad lo plantea en términos parecidos cuando advierte que al distribuir la funcionalidad en múltiples servicios la complejidad aumenta en orquestación, monitoreo y comunicación, y que aspectos como la detección de servicios, el manejo de fallos de red o la consistencia de datos exigen un esfuerzo de arquitectura adicional (Corporación Escuela Tecnológica del Oriente, 2025). Esa advertencia fue, en la práctica, la lista de problemas que hubo que resolver.

Esa distinción es la que ordena todo el diseño que viene después. Dentro del nodo se usa un candado, y entre nodos se usan mensajes idempotentes y compensación.

Metas del Sistema

Las metas traducen los objetivos a resultados comprobables sobre el prototipo. Cada una se declara aprobada o reprobada en la sección de evaluación, con el dato al lado, de modo que no queden como aspiraciones generales.

Tabla 5

Metas del Sistema y sus Criterios de Aprobación

Meta	Criterio de aprobación	Requisito
M1. Concurrencia segura	Cero sobregiros de cupo con múltiples hilos compitiendo	RD-NF04
M2. Respuesta bajo carga	Percentil 95 de latencia menor o igual a 300 ms	RD-NF01
M3. Escalado horizontal	Al menos 70 % del incremento lineal al pasar de 1 a 4 nodos	RD-NF02
M4. Continuidad ante partición	Cero apuestas rechazadas mientras el nodo está desconectado	RD-NF05

Meta	Criterio de aprobación	Requisito
M5. Convergencia	Cero pérdidas y cero duplicados al sincronizar	RD-NF06
M6. Aislamiento de fallos	La caída de un nodo no degrada de forma medible a los demás	RD-NF09
M7. Corte temporal exacto	Cero apuestas admitidas después del sello de cierre	RD-NF11

Nota. Elaboración propia.

Estrategia de Trabajo y Cronograma

La entrega es individual, así que los roles que normalmente se reparten en un equipo se asumieron de forma secuencial por una sola persona. Aun así se documentan, ya que cada rol implica un criterio de decisión distinto y forzar el cambio de sombrero evita que el diseño y su evaluación se hagan con la misma mirada complaciente. En la práctica esto sirvió, porque el rol de evaluador fue el que encontró que la arquitectura propuesta no escalaba como se había prometido, algo que el rol de diseñador había dado por hecho.

Tabla 6

Roles Asumidos y Productos Asociados

Rol	Responsabilidad	Producto
Líder de proyecto	Alcance, cronograma y verificación final de los entregables	Plan de proyecto
Analista de requisitos	Levantamiento, priorización y definición de métricas	Tablas 2 y 3
Diseñador de arquitectura	Descomposición en nodos, contratos y selección de patrones	Figuras 1 a 3 y Tabla 7
Desarrollador	Implementación de los cinco tipos de nodo	Prototipo y repositorio
Evaluador de calidad	Pruebas de carga y robustez, análisis de cuellos de botella	Tablas 8 a 11
Documentador	Redacción en normas APA, presentación y guion	Informe y sustentación

Nota. Elaboración propia. Como evidencia del trabajo organizado se conserva el historial de confirmaciones fechadas del repositorio.

El cronograma se organizó en diez actividades entre el 12 y el 26 de septiembre de 2026. Las primeras tres corresponden al análisis, la definición de requisitos y la planificación. Las tres siguientes abarcan el modelado en UML, la implementación del prototipo y las pruebas de carga y robustez. Las últimas cuatro cubren la redacción del informe, la presentación, la grabación del video y la verificación final.

Modelado de la Arquitectura Mediante UML

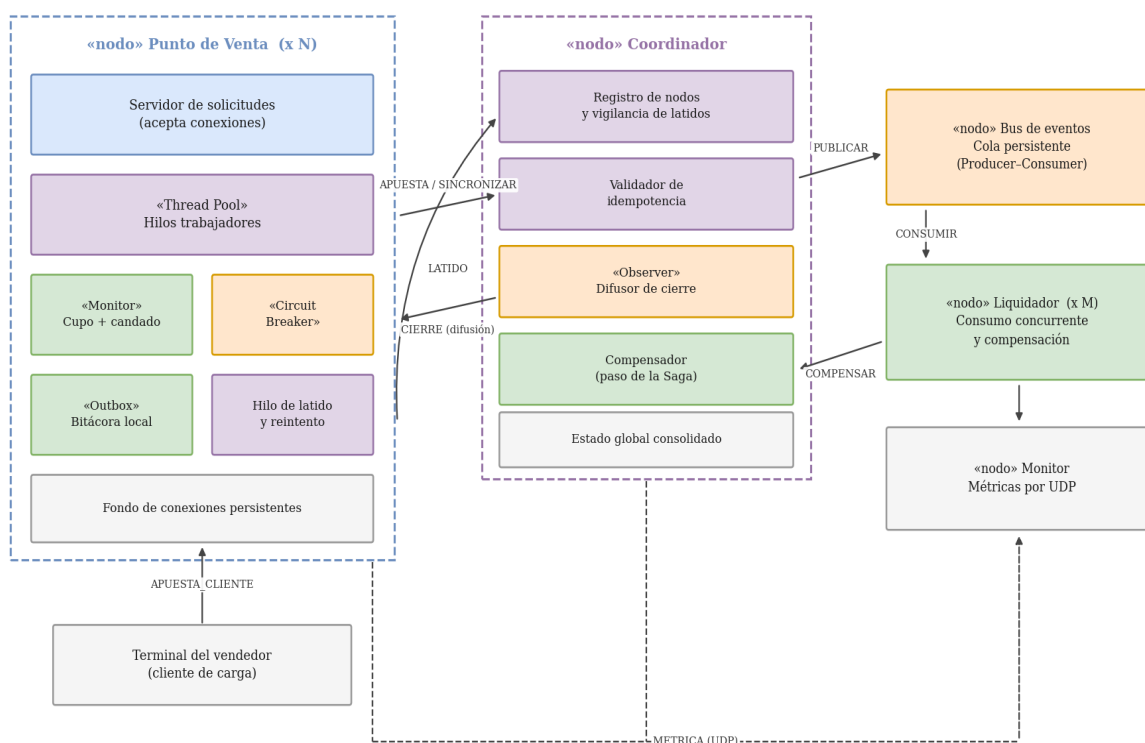
Se elaboraron los tres diagramas solicitados.

Diagrama de Componentes

El primer diagrama muestra cómo quedó repartida la funcionalidad y qué patrón vive en cada pieza. Más allá de la lista de cajas, hay algo que se ve de una sola mirada. El nodo de punto de venta concentra casi toda la lógica de tolerancia a fallos, mientras que el coordinador concentra la lógica de coherencia. Esa separación responde a que cada uno enfrenta un problema diferente.

Figura 1

Diagrama de Componentes del Sistema Distribuido



Nota. Los estereotipos entre comillas angulares indican el patrón de diseño aplicado en cada componente. Las flechas punteadas representan comunicación asíncrona sin confirmación. Elaboración propia.

Las métricas viajan por un canal aparte y sin confirmación, y eso fue una decisión deliberada, ya que la observabilidad no debería competir con el registro de apuestas por las

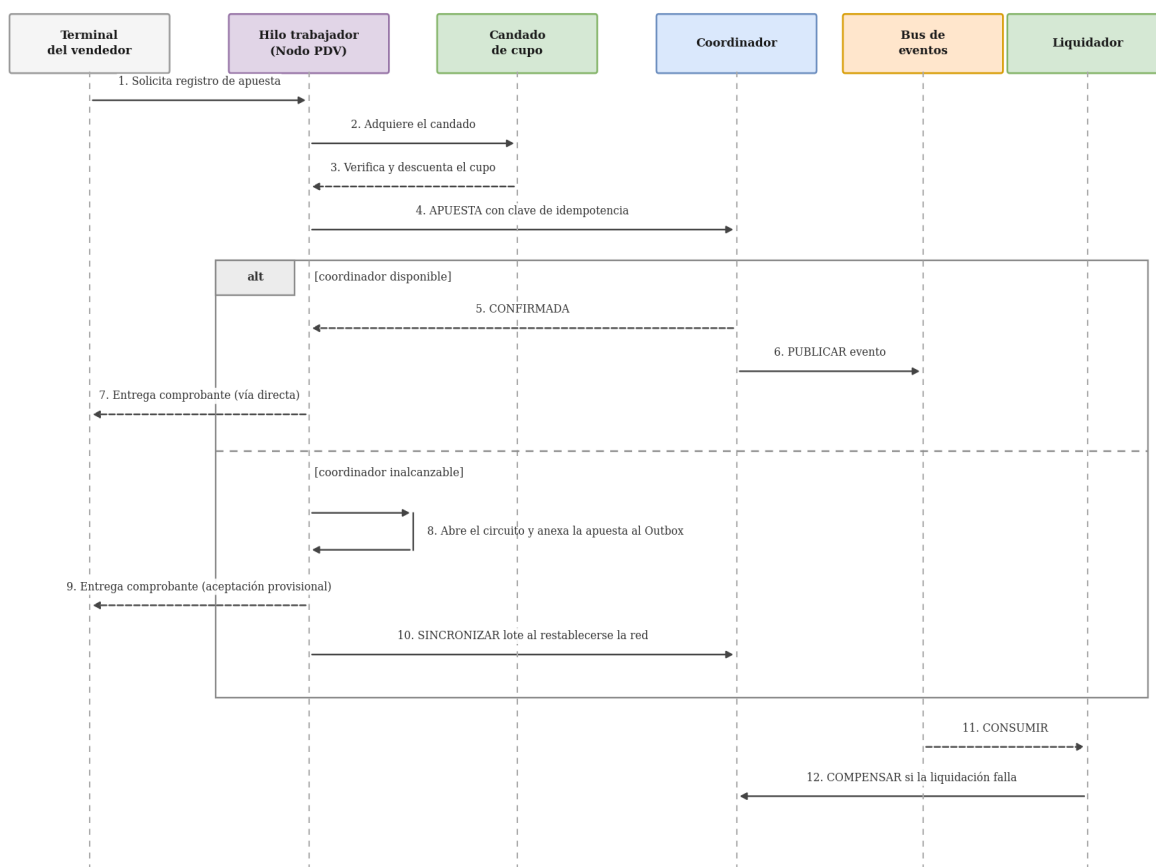
mismas conexiones. Además, perder una métrica ocasional no es grave, mientras que perder una apuesta sí lo sería.

Diagrama de Secuencia

El segundo diagrama recorre el flujo crítico, que es el registro de una apuesta. Se eligió ese recorrido porque en él se concentran los requisitos más exigentes y porque es donde se ven trabajando juntos casi todos los patrones.

Figura 2

Diagrama de Secuencia del Registro de una Apuesta



Nota. Las flechas punteadas representan mensajes de retorno. El fragmento alt distingue los dos caminos que sigue una misma apuesta según el estado de la red. Elaboración propia.

El fragmento alternativo es la parte interesante. La misma apuesta sigue dos caminos según el estado de la red, pero en ambos el cliente recibe una respuesta afirmativa. Cuando hay

conexión, la apuesta se confirma contra el coordinador y se publica el evento. Cuando no la hay, el circuito se abre, la apuesta se guarda en la bitácora local y el vendedor recibe igual su comprobante. Desde la perspectiva del punto de venta la operación es coherente en los dos casos, ya que el cupo se descontó localmente antes de intentar la red.

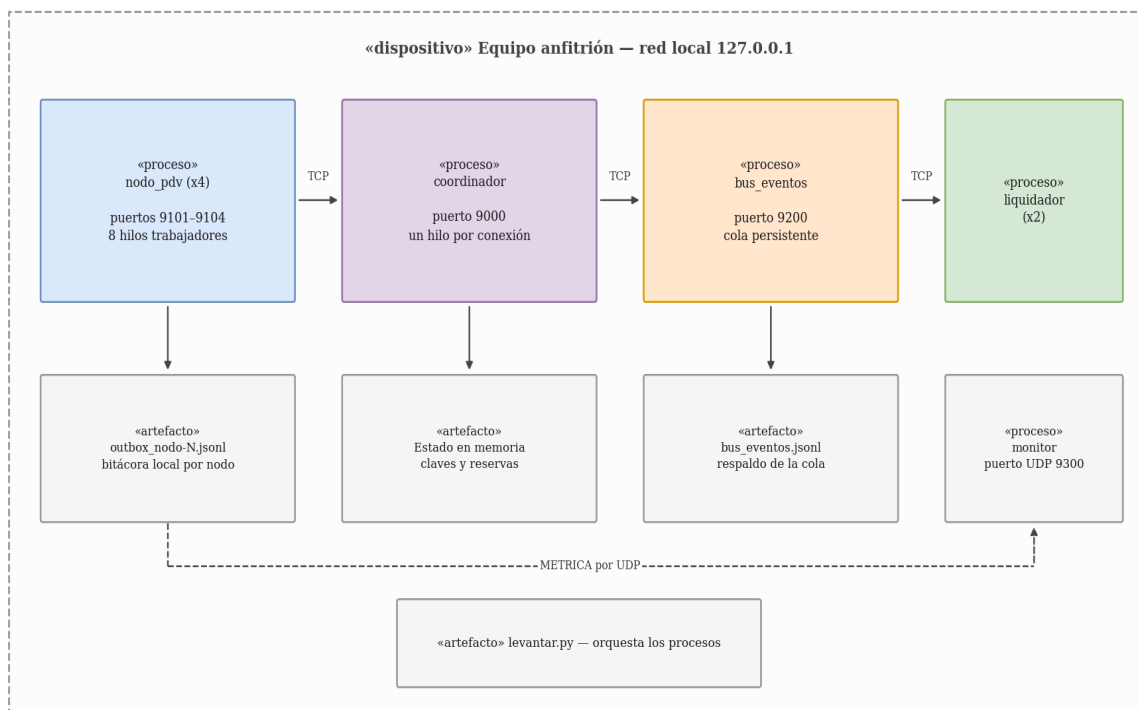
Ese detalle del orden importa más de lo que parece. Si se publicara primero en el bus y la operación local fallara después, el sistema quedaría anunciando algo que nunca pasó.

Diagrama de Despliegue

El tercer diagrama ubica cada componente en la infraestructura y muestra los puertos que ocupa. Aquí se hace evidente lo que se dijo en los supuestos, y es que cada pieza es un proceso del sistema operativo con su propio espacio de memoria, así que la comunicación entre ellas es siempre por red y puede fallar de verdad.

Figura 3

Diagrama de Despliegue de la Red Distribuida



Nota. Cada componente corre como un proceso independiente. La bitácora de salida es local a cada nodo, y eso es lo que permite que un punto de venta siga operando cuando queda aislado. Elaboración propia.

También se ve que el estado del coordinador vive en memoria. Esa decisión simplificó el prototipo, pero tiene consecuencias que se discuten en la evaluación.

Selección y Justificación de Patrones de Diseño

Criterio de Selección

Se mantuvo el criterio de la unidad anterior y se aplicó sin excepciones. Cada patrón debe resolver un requisito que ya estaba documentado, y para cada uno se escribió primero qué falla concreta ocurriría si no estuviera. Si esa falla no se puede nombrar, entonces el patrón sobra.

La guía de aprendizaje define un patrón como una solución general y reutilizable a un problema común de diseño, derivada de la experiencia acumulada de la industria, y distingue entre los patrones que se aplican a nivel de clases y los que operan a un nivel más alto dictando la organización global de los componentes (Corporación Escuela Tecnológica del Oriente, 2025). En este trabajo se usaron de los dos tipos, ya que un entorno distribuido introduce problemas que el catálogo clásico no contempla (Castro, 2015).

Se aplicaron diez patrones. Cuatro vienen de la Unidad 1 y acá pasan de la propuesta a la implementación real; los otros seis son nuevos y propios de lo distribuido. La Tabla 7 resume la trazabilidad completa.

Tabla 7

Trazabilidad Entre Patrones, Fallas Evitadas y Requisitos

Patrón	Falla que evita	Requisito	Origen
Thread Pool	Crear un hilo por solicitud agota la memoria en el pico de cierre	RD-F03, RD-NF03	Nuevo
Monitor o exclusión mutua	Dos hilos descuentan sobre el mismo saldo y se sobregira el cupo	RD-F04, RD-NF04	Nuevo
Outbox con sincronización diferida	El punto de venta queda inservible al perder la conexión	RD-F06, RD-F08	Heredado
Clave de idempotencia	El reenvío tras la reconexión crea apuestas duplicadas	RD-F05, RD-F09	Heredado
Observer distribuido	Cada nodo consulta el estado del sorteo y satura al coordinador	RD-F10, RD-F11	Nuevo
Saga con compensación	Queda cupo comprometido sin apuesta asociada cuando falla un paso	RD-F14	Heredado
Circuit Breaker	Un coordinador lento agota los hilos del nodo y tumba también lo local	RD-F15, RD-NF10	Heredado

Patrón	Falla que evita	Requisito	Origen
Heartbeat	Un nodo caído se confunde con uno lento y nadie lo detecta	RD-F01, RD-F02	Nuevo
Producer–Consumer	La liquidación bloquea el registro en el momento de mayor carga	RD-F12, RD-F13	Nuevo
Mediator	Los nodos se comunican entre sí y las rutas crecen de forma cuadrática	RD-F16	Nuevo

Nota. Elaboración propia. Los patrones marcados como heredados fueron justificados en la Unidad 1 y en este trabajo pasan de la propuesta a la implementación verificada.

Justificación de los Patrones Principales

Thread Pool y exclusión mutua. Estos dos van juntos porque resuelven las dos caras del mismo problema dentro del nodo. El conjunto de hilos permite atender varias apuestas a la vez sin crear un hilo nuevo por cada solicitud, lo cual en el pico de cierre agotaría la memoria. Ahora bien, apenas hay concurrencia aparece el riesgo del cupo compartido, y ahí entra el candado. La sección crítica se mantuvo lo más corta posible a propósito, o sea, dentro del candado solo ocurre la verificación y el descuento, nada de entrada o salida, ya que cualquier operación lenta dentro de esa sección se convierte en una fila de hilos esperando.

Outbox con clave de idempotencia. Estos también forman pareja. La bitácora de salida permite que el punto de venta siga vendiendo cuando el coordinador no responde, pero abre un problema nuevo, ya que al reconectar hay que reenviar todo, y un reenvío sin protección crea apuestas duplicadas. La clave de idempotencia, generada en el nodo de origen antes del primer intento, es lo que hace seguro ese reenvío. El coordinador reconoce la clave repetida y devuelve el resultado original en lugar de registrar una apuesta nueva. Sin ese par, la tolerancia a fallos se pagaría con dinero descuadrado.

Circuit Breaker. El nodo depende de un coordinador que no controla. Sin protección, cada solicitud esperaría el tiempo de espera completo antes de rendirse, y con suficientes solicitudes en paralelo los hilos del nodo se agotarían atendiendo llamadas condenadas al fracaso. Es decir, la caída del coordinador terminaría tumbando también la parte local, que era lo que el Outbox intentaba salvar. El interruptor detecta la tasa de fallos, corta el circuito y responde de inmediato. Además, se conservó un ajuste de la unidad anterior que resultó

importante, y es que el rechazo por reglas de negocio, como una apuesta posterior al cierre, no cuenta como fallo de infraestructura, ya que es una respuesta válida del sistema y no una avería.

Observer distribuido. El cierre de sorteo podría resolverse haciendo que cada nodo pregunte al coordinador si el sorteo sigue abierto, pero eso multiplicaría el tráfico justo en el minuto de mayor carga. En cambio, difundir el cierre una sola vez hacia todos los nodos suscritos invierte la relación, porque el nodo se entera y rechaza localmente. Durante las pruebas esto tuvo un efecto que no se había previsto y que se comenta en la evaluación.

Producer–Consumer y Saga. La liquidación de premios no tiene por qué ocurrir dentro del registro de la apuesta. Separarla mediante una cola permite que el registro responda rápido y que la liquidación se procese después, con varios consumidores en paralelo. El problema es que si la liquidación falla ya no hay una transacción que revertir, porque el registro ocurrió en otro momento y en otro proceso. La Saga resuelve eso con compensación, o sea que el liquidador emite un evento que revierte la reserva asociada. No es lo mismo que deshacer, pero es lo único posible cuando los pasos viven separados (Blas et al., 2019).

Heartbeat y Mediator. El latido periódico es lo que convierte el silencio en información. Sin él, un nodo caído y un nodo lento son exactamente lo mismo para el resto del sistema. El Mediator, por su parte, es la decisión de que todo pase por el coordinador en lugar de que los nodos se hablen entre sí. Gana claridad y control de duplicados, aunque, como se verá, concentra el cuello de botella.

Patrones Descartados

Un catálogo de patrones no es una lista de compras, así que también hay que dejar constancia de lo que quedó por fuera. Se evaluó incorporar un servicio de descubrimiento independiente y se descartó, porque con un solo coordinador el registro directo cumple la misma función sin agregar otro punto que pueda fallar.

También se consideró replicar el coordinador para eliminar el punto único de fallo. Se descartó para este prototipo por una razón honesta, y es que el coordinador guarda su estado en memoria, así que replicarlo exige antes mover ese estado a un almacenamiento compartido, y eso

convertiría el ejercicio en otro proyecto. Queda planteado como trabajo futuro y, de hecho, la evaluación muestra por qué sería lo primero que habría que hacer.

Por último, se descartó cifrar la comunicación entre nodos. En una operación real sería obligatorio, pero acá habría añadido complejidad de configuración sin incidir en el comportamiento concurrente ni en la tolerancia a fallos, que son los aspectos que el prototipo pone a prueba.

Implementación y Simulación del Prototipo

Alcance del Prototipo

El prototipo no intenta cubrir los diecinueve requisitos funcionales, ya que eso sería un producto y no una prueba de concepto. Se implementó de principio a fin el recorrido completo de una apuesta en el canal físico, incluyendo sus caminos de falla, porque es ahí donde viven los patrones más importantes.

Buena parte del valor del prototipo está en poder romperlo a propósito. Sin un fallo real no habría nada que mostrar, así que se incluyeron mecanismos para provocar la caída del coordinador, la partición de red y el fallo de la liquidación.

Tecnologías y Estructura

Todo se construyó en Python 3.11 usando únicamente la biblioteca estándar. La decisión fue deliberada, porque sin dependencias externas cualquiera puede levantar la red completa con un solo comando y sin instalar nada, lo cual atiende directamente al requisito RD-NF13. Los hilos se manejan con el módulo de concurrencia estándar y la comunicación se hace con sockets TCP, sobre un protocolo de texto en formato JSON delimitado por saltos de línea.

Se eligió un protocolo de texto en lugar de uno binario por una razón práctica, y es que así se puede abrir el tráfico y leerlo mientras el sistema corre.

El repositorio se organizó separando los cinco tipos de nodo, una carpeta común con las piezas transversales, una carpeta de pruebas y otra de resultados. Cada patrón vive en un archivo identificable, y el archivo léame incluye un mapa que indica dónde encontrar cada uno. La estructura de carpetas refleja la arquitectura y no al revés (Fontela, 2009).

Evidencia de los Patrones en el Código

El candado del cupo es la pieza más pequeña y a la vez la más crítica. Protege solo dos operaciones, la verificación y el descuento, y todo lo demás queda fuera para no alargar la espera de los otros hilos.

La bitácora de salida se escribe línea por línea en un archivo propio de cada nodo, y un hilo aparte la reintenta cada segundo. Cuando la sincronización tiene éxito, el coordinador responde por cada clave si quedó confirmada o si ya existía, y solo entonces se limpian del archivo las que efectivamente llegaron. Ese orden evita perder apuestas ante una sincronización parcial.

El interruptor de circuito envuelve las llamadas del nodo hacia el coordinador y lleva su propio conteo de fallos dentro de una ventana de tiempo. Durante las pruebas se comprobó que abre correctamente, y el efecto se ve en los tiempos. La primera solicitud que encuentra el coordinador caído espera el tiempo completo, pero las siguientes fallan de inmediato.

Un hallazgo del desarrollo merece mencionarse. Al probar el reinicio del coordinador se descubrió que los nodos no volvían a registrarse, ya que el registro solo ocurría al arrancar. Era un vacío real, porque el sistema toleraba la caída de un nodo pero no la del propio coordinador. Se corrigió haciendo que el latido de un nodo desconocido funcione como registro implícito, con lo cual la red se recompone sola.

Repositorio y Puesta en Marcha

El código fuente está disponible en el repositorio <https://github.com/naciontek/UTO-AS-U2-AA4>. El archivo léame contiene las instrucciones de ejecución, el mapa de patrones por archivo y el procedimiento para reproducir cada escenario de falla. La red completa se levanta con un solo comando y se detiene de forma ordenada.

La presentación grabada se encuentra publicada en YouTube, en modo no listado, y su enlace se incluye en el espacio de entrega del aula virtual.

Evaluación del Desempeño y Ajustes

Método de Medición

Se construyó un cliente de carga que genera apuestas concurrentes contra uno o varios nodos y reporta el throughput, la latencia promedio y el percentil 95. Todas las corridas que se reportan acá usan el mismo equipo, la misma configuración de cupo y el mismo número de apuestas, de modo que las comparaciones tengan sentido.

Hay una limitación que conviene aclarar. Al correr todos los nodos en una sola máquina, los procesos compiten por los mismos núcleos. Eso no invalida las mediciones de robustez ni las de concurrencia, pero sí acota lo que se puede concluir sobre escalabilidad, y así se interpreta más adelante.

Escalabilidad Horizontal y Cuello de Botella

La primera prueba consistió en enviar cinco mil apuestas con cien hilos de cliente, variando el número de nodos de punto de venta de uno a cuatro. Si la arquitectura escalara bien, el throughput debería crecer al agregar nodos. El resultado inicial fue el que aparece en la columna izquierda de la Tabla 8, y no fue el esperado.

El throughput se quedó estancado alrededor de mil apuestas por segundo sin importar cuántos nodos hubiera. De hecho, con tres nodos fue levemente peor que con uno. Ese resultado obligó a parar y buscar la causa.

El diagnóstico fue que los nodos no eran el límite. Todas las apuestas terminan pasando por un único proceso coordinador, y además cada apuesta abría una conexión TCP nueva hacia él. Agregar nodos solamente aumentaba la cantidad de conexiones que ese proceso tenía que aceptar, establecer y cerrar. O sea, el patrón Mediator, que había sido una buena decisión para la claridad del diseño, se convirtió en el cuello de botella del sistema.

Ajuste Aplicado y Medición Posterior

El ajuste consistió en que cada nodo mantenga un pequeño fondo de conexiones persistentes hacia el coordinador, del mismo tamaño que su conjunto de hilos, en lugar de abrir y

cerrar una conexión por cada apuesta. Esto obligó también a modificar el coordinador, ya que antes cerraba la conexión después de responder un solo mensaje.

La misma prueba se repitió con el ajuste aplicado. Los resultados aparecen en la Tabla 8 y la comparación completa se grafica en la Figura 4.

Tabla 8

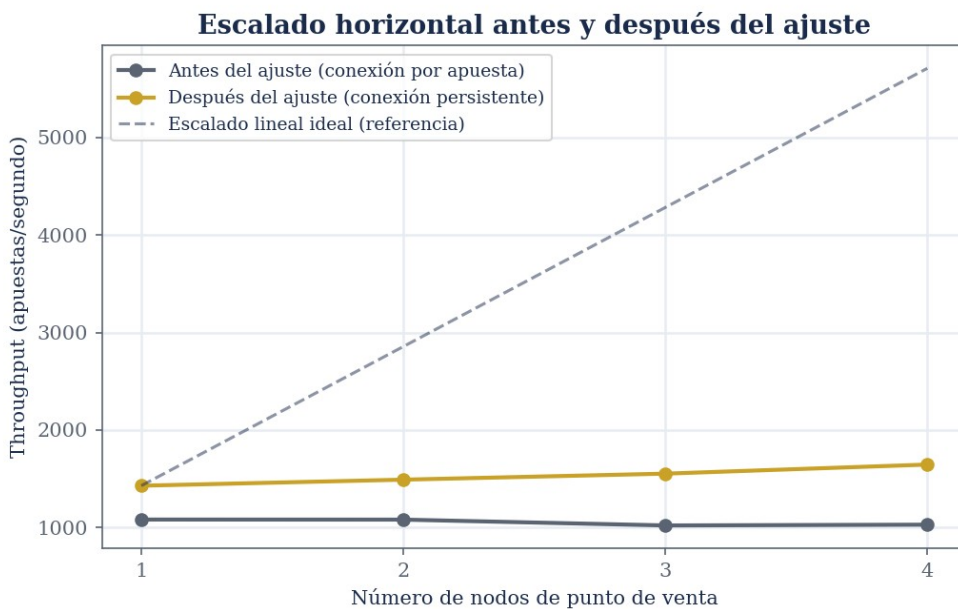
Throughput y Latencia Según el Número de Nodos, Antes y Después del Ajuste

Nodos	Throughput antes (ap/s)	Throughput después (ap/s)	Latencia p95 después (ms)	Fallidas
1	1.080,5	1.427,7	74,9	0
2	1.079,7	1.489,7	89,9	0
3	1.019,9	1.551,4	80,8	0
4	1.027,8	1.644,5	69,1	0

Nota. Cada corrida envió 5.000 apuestas con 100 hilos de cliente concurrentes. El ajuste consistió en reemplazar la conexión por apuesta por un fondo de conexiones persistentes hacia el coordinador. Elaboración propia.

Figura 4

Escalado Horizontal Antes y Después del Ajuste



Nota. La línea punteada superior representa el escalado lineal ideal tomando como base el resultado con un solo nodo. Elaboración propia.

El ajuste funcionó y la mejora es clara. El throughput con un solo nodo subió un 32 % y, más importante, la curva ahora sí crece al agregar nodos, cosa que antes no hacía. La latencia también mejoró alrededor de un 25 %, ya que se eliminó el costo de establecer una conexión nueva en cada operación.

Sin embargo, la meta M3 no se cumplió. Pasar de uno a cuatro nodos mejoró el throughput apenas un 15 %, muy lejos del 70 % del incremento lineal que se había comprometido en RD-NF02. En la Figura 4 se alcanza a ver la distancia que quedó respecto de la línea ideal.

La explicación es la misma de antes, solo que ahora medida. El coordinador sigue siendo un único proceso con estado en memoria, y todas las apuestas siguen serializándose ahí. El fondo de conexiones eliminó el costo de establecer conexiones, que era el desperdicio evidente, pero no eliminó la serialización, que es estructural. Mientras el coordinador no se pueda replicar, agregar nodos de punto de venta va a seguir dando rendimientos decrecientes.

De ese resultado salió la lección más útil de todo el ejercicio. La guía advierte que los microservicios permiten escalar solo los servicios que lo requieren, siempre que se pueda identificar cuál es el que se convirtió en cuello de botella (Corporación Escuela Tecnológica del Oriente, 2025). Acá el componente que había que escalar era justo el que, por su diseño, no se podía escalar.

Punto de Saturación de los Hilos

La segunda prueba midió el efecto del tamaño del conjunto de hilos dentro de un solo nodo. Se lanzaron tres mil apuestas con ciento cincuenta hilos de cliente contra un nodo que se reinició con distintas configuraciones.

Tabla 9

Throughput y Latencia Según el Número de Hilos Trabajadores del Nodo

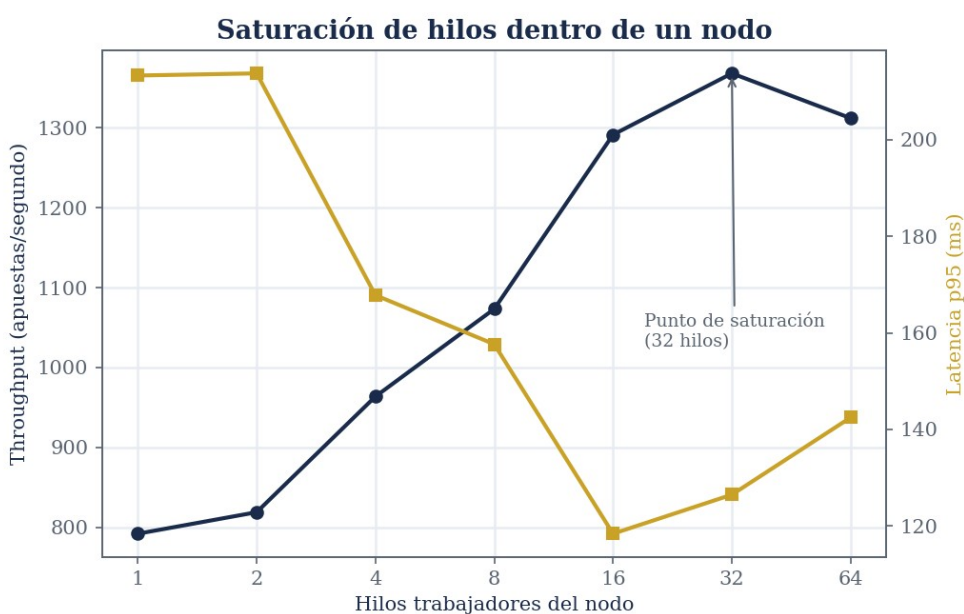
Hilos	Throughput (ap/s)	Latencia p95 (ms)
1	792,0	213,3
2	818,9	213,8
4	963,7	167,8

Hilos	Throughput (ap/s)	Latencia p95 (ms)
8	1.073,2	157,7
16	1.291,2	118,4
32	1.368,0	126,5
64	1.311,9	142,5

Nota. Cada corrida envió 3.000 apuestas con 150 hilos de cliente. Ninguna apuesta falló en ninguna de las siete configuraciones. Elaboración propia.

Figura 5

Punto de Saturación del Conjunto de Hilos



Nota. El throughput crece hasta los 32 hilos y decae con 64, lo cual marca el punto de saturación. Elaboración propia.

La curva se comporta como predice la teoría, así que el requisito RD-NF03 queda cerrado con un dato medido. El throughput sube de forma sostenida hasta los treinta y dos hilos, donde alcanza su máximo, y con sesenta y cuatro empieza a caer. La latencia acompaña el mismo movimiento, porque baja hasta los dieciséis hilos y vuelve a subir después.

La lectura es que a partir de cierto punto los hilos adicionales ya no encuentran trabajo libre, sino que compiten por el candado del cupo y por el cambio de contexto del procesador. Es el mismo fenómeno que se describió en la sección sobre comunicación entre hilos, solo que

ahora con números. El prototipo quedó configurado en ocho hilos por defecto, un valor conservador; para esta máquina el óptimo estaría entre dieciséis y treinta y dos.

Pruebas de Robustez

Las pruebas de carga dicen qué tan rápido va el sistema cuando todo funciona, y las de robustez dicen qué pasa cuando algo se rompe, que al final es lo que más interesa en el canal físico.

El primer escenario fue la partición de red. Con la red operando se detuvo el proceso del coordinador y se siguió enviando carga contra un nodo. El resultado aparece en la Tabla 10.

Tabla 10

Resultados de las Pruebas de Robustez

Escenario	Resultado observado	Meta
Carga con el coordinador caído	400 de 400 apuestas aceptadas, cero rechazos por la desconexión	M4
Preservación local	Las 400 apuestas quedaron en la bitácora de salida del nodo	M4
Sincronización al reconectar	400 confirmadas en 1,16 s, cero pérdidas y cero duplicados	M5
Reenvío de una clave repetida	El coordinador devolvió el resultado original y la contó como duplicada	M5
Aislamiento de la caída	Los nodos siguieron atendiendo durante toda la desconexión	M6
Cierre de sorteo difundido	500 aceptadas antes del cierre y 300 rechazadas después, ninguna admitida	M7
Integridad del cupo	Cero sobregiros; al agotarse el cupo el nodo rechazó correctamente	M1

Nota. Todas las pruebas se ejecutaron sobre la red completa levantada con la configuración por defecto. Elaboración propia.

Más allá de las cifras, de esa corrida salieron dos observaciones.

La primera tiene que ver con el Circuit Breaker. Durante la partición, la latencia máxima registrada fue de 1.531 milisegundos, que coincide casi exactamente con el tiempo de espera configurado, pero el promedio quedó en 90 milisegundos. Eso significa que solo las primeras solicitudes esperaron el tiempo completo; en cuanto el interruptor acumuló los fallos suficientes, abrió el circuito y las demás cayeron a la bitácora de inmediato sin volver a intentar la red. O sea

que el patrón se alcanza a ver funcionando en la distribución de los tiempos, y no solo en el código.

La segunda tiene que ver con el cierre de sorteo. Después de difundir el cierre, las trescientas apuestas siguientes fueron rechazadas, pero el coordinador registró cero rechazos. Al principio pareció un error, y en realidad es la prueba de que el Observer distribuido está haciendo su trabajo, porque los nodos ya sabían del cierre y rechazaron localmente, sin molestar al coordinador. Eso era justo lo que se buscaba al difundir el evento en lugar de hacer que cada nodo pregunte.

Sobre la integridad del cupo apareció además una comprobación que no estaba planeada. En una de las corridas iniciales, con un cupo bajo, empezaron a aparecer rechazos masivos que al principio se interpretaron como una falla. Al revisarlo resultó que el cupo del nodo se había agotado de verdad, o sea, el mecanismo de exclusión mutua estaba funcionando y protegiendo el saldo tal como se pedía en RD-F04.

Cumplimiento de las Metas

La Tabla 11 cierra la evaluación declarando el estado de cada meta con el dato al lado.

Tabla 11

Estado de Cumplimiento de las Metas del Sistema

Meta	Estado	Evidencia
M1. Concurrencia segura	Cumplida	Cero sobregiros en todas las corridas
M2. Respuesta bajo carga	Cumplida	Percentil 95 entre 69 y 90 ms, muy por debajo de los 300 ms
M3. Escalado horizontal	No cumplida	Mejora del 15 % de 1 a 4 nodos frente al 70 % comprometido
M4. Continuidad ante partición	Cumplida	400 de 400 apuestas aceptadas sin coordinador
M5. Convergencia	Cumplida	Cero pérdidas y cero duplicados al sincronizar
M6. Aislamiento de fallos	Cumplida	Los nodos operaron durante toda la caída del coordinador
M7. Corte temporal exacto	Cumplida	Ninguna apuesta admitida después del sello de cierre

Nota. Elaboración propia a partir de las corridas reportadas en las Tablas 8, 9 y 10.

Seis de las siete metas se cumplieron. La que falló es la de escalabilidad horizontal, y falló por una razón que quedó identificada y medida, así que se puede proponer una corrección concreta y no solo una intención.

Conclusiones

El ejercicio confirma que un patrón solo se puede evaluar cuando está implementado y medido. En la Unidad 1 los ocho patrones se justificaron con argumentos razonables y todos parecían acertados. Acá, al ponerlos a correr, siete se comportaron como se esperaba y uno, el Mediator, resultó ser al mismo tiempo la decisión que daba claridad al diseño y la que limitaba su rendimiento. Esa doble cara no se veía en el papel.

La separación entre lo que ocurre dentro de un nodo y lo que ocurre entre nodos resultó ser la distinción más útil de todo el trabajo. Dentro del nodo el problema es la memoria compartida y se resuelve con un candado; entre nodos no hay memoria compartida y un nodo caído es indistinguible de uno lento, así que hacen falta idempotencia, latidos y compensación. Confundir los dos planos lleva a aplicar la herramienta equivocada, y buena parte de los patrones distribuidos existen justamente porque los mecanismos de concurrencia local no sirven cuando hay una red de por medio.

La tolerancia a fallos se comportó mejor de lo esperado. El sistema soportó la caída completa del nodo central sin rechazar una sola apuesta y convergió sin pérdidas ni duplicados en poco más de un segundo. Además, en el camino apareció un vacío que no se había previsto, que era la incapacidad de recuperarse del reinicio del propio coordinador, y corregirlo dejó la red más robusta de lo que estaba diseñada originalmente.

Sobre la escalabilidad hay poco que matizar. La arquitectura no escaló como se había prometido, y el motivo no estuvo en la implementación sino en una decisión de diseño tomada mucho antes, que fue guardar el estado del coordinador en memoria. Eso lo vuelve imposible de replicar, y un componente que no se puede replicar fija el techo de todo el sistema por muchos nodos que se le agreguen alrededor. Es un buen recordatorio de que las decisiones arquitectónicas se toman al principio y cambiarlas después resulta costoso.

Quedan limitaciones claras. Todos los nodos corrieron en una sola máquina, así que las cifras absolutas de throughput no se pueden extrapolar a una red real, y las pruebas cubrieron un solo recorrido funcional. Además, la comunicación entre nodos se dejó sin cifrar, cosa que en una operación regulada no sería aceptable.

Como trabajo futuro se propone, en este orden: mover el estado del coordinador a un almacenamiento compartido para poder replicarlo, que es lo único que destrabaría la meta de escalabilidad; distribuir los nodos en máquinas separadas para medir el efecto real de la latencia de red; y agregar cifrado en tránsito junto con autenticación entre nodos.

Referencias

- Blas, M. J., Leone, H. P., y Gonnet, S. M. (2019). Modelado y verificación de patrones de diseño de arquitectura de software para entornos de computación en la nube. En Proceedings del Congreso Nacional de Ingeniería Informática/Sistemas de Información.
- Castro, L. M. (2015). Arquitectura del software. Cengage Learning.
- Corporación Escuela Tecnológica del Oriente. (2025). Arquitectura de software: Principios, patrones y evaluación [Guía de aprendizaje, Unidad 2]. Corporación Escuela Tecnológica del Oriente.
- Fontela, C. (2009, 23 de junio). Modularidad, cohesión y acoplamiento. Cysingsoft.
<https://cysingsoft.wordpress.com/2009/06/23/modularidad-cohesion-y-acoplamiento-carlos-fontela/>
- Pardo Moreno, J. C., y Otavo Piraban, A. (2025). Modelos de procesos de desarrollo aplicado a un proyecto de arquitectura de software. Tecnología, Investigación y Academia, 12(1), 150–161. <https://doi.org/10.53095/ti.22231>
- Red Hat. (2023). ¿Qué son los microservicios? Ventajas de la arquitectura de microservicios. Red Hat. <https://www.redhat.com/es/topics/microservices/what-are-microservices>
- Reynoso, C. B. (2004). Introducción a la arquitectura de software. Universidad de Buenos Aires.